



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Proyecto Bases de Datos

Sistema de Comandas Profesional

Materia: Bases de Datos

Profesor: Gabriel Hurtado Avilés

Alumnos:

Damas Garnica Adrian

Hernandez Barrios Samuel Rodrigo

Grupo: 3CV4

Carrera: Ingeniería en Sistemas Computacionales

Resumen

En este documento se recopila, de manera formal, el desarrollo de la base de datos del Sistema de Comandas Profesional realizado a lo largo de las prácticas de la materia de Bases de Datos, desde el análisis del caso de estudio y el diseño de sus modelos hasta su implementación, población y comprobación en PostgreSQL.

Abstract

This document formally compiles the development of the Sistema de Comandas Profesional database, carried out throughout the practices of the Databases course, from the analysis of the case study and the design of its models to its implementation, population, and verification in PostgreSQL.

Índice

1 Introducción **1**
1.1 Planteamiento del problema 1
1.2 Justificación 1
1.3 Objetivo general 2
1.4 Objetivos específicos 2
1.5 Estado del arte 3
1.6 Alcance 3
1.7 Organización del documento 4

2 Marco teórico **5**
2.1 Dato, información y base de datos 5
2.2 Sistema gestor de bases de datos 5
2.3 Ciclo de vida de una base de datos 6
2.4 Modelo entidad-relación y su extensión 7
2.5 Modelo relacional 8
2.6 Implementación en un gestor relacional 8
2.6.1 Restricciones de dominio 8
2.6.2 Integridad referencial 9
2.6.3 Disparadores y vistas 9
2.6.4 Transacciones 9
2.6.5 Control de acceso 9
2.6.6 Carga masiva de datos 10

3 Metodología y entorno de trabajo **11**
3.1 Desarrollo por etapas 11
3.2 Control de versiones 11
3.3 Contenedores y gestor de bases de datos 12
3.4 Automatización y organización del repositorio 13

Índice de figuras

1 Flujo de ramas del repositorio, con ejemplos de las prácticas 4 y 5 12

Índice de tablas

1 Trabajos relacionados con el proyecto 3
2 Características de un sistema gestor de bases de datos 6
3 Etapas del ciclo de vida trabajadas en cada práctica 11
4 Configuración del contenedor del gestor 13
5 Organización del repositorio 14

1 Introducción

En esta sección se presenta el problema que da origen al proyecto, las razones que justifican el diseño de una base de datos para atenderlo, los objetivos que guiaron su desarrollo, los trabajos relacionados que se revisaron y el alcance del documento.

1.1 Planteamiento del problema

Los salones de eventos que ofrecen servicio de banquetes gestionan la toma y el seguimiento de las comandas, es decir, los pedidos de alimentos y bebidas que realiza cada mesa durante un evento, con comandas de papel, hojas de cálculo o aplicaciones genéricas que no fueron diseñadas para este giro. Con esta forma de operar, la información de los pedidos se duplica o se pierde entre el área de meseros, la cocina y la caja; no queda un registro histórico confiable de lo que se sirvió en cada evento ni de su costo; el inventario de insumos se controla de manera aproximada, lo que provoca faltantes o compras innecesarias; y la cuenta final de cada evento se elabora conciliando a mano varias comandas, lo que la vuelve lenta y propensa a errores.

El salón que se tomó como caso de estudio atiende alrededor de quince eventos al mes, con unas veinte mesas por evento y dos o tres cuentas por mesa. Para atenderlo, el equipo contaba con un sistema de comandas desarrollado en un semestre anterior en la materia de Sistemas Operativos: una interfaz de terminal escrita en C que guardaba la información en archivos de texto. Ese sistema carecía de relaciones bien definidas, de normalización y de reglas de integridad, de modo que cualquier cambio en el negocio obligaba a modificar el programa y nada impedía que los archivos quedaran en un estado inconsistente.

Por lo anterior, el problema que se atiende no consiste solamente en digitalizar la toma de comandas, sino en diseñar e implementar, con una metodología formal, la base de datos que sustente una versión profesional de ese sistema, capaz de representar eventos, mesas, cuentas, comandas, productos, ingredientes, personal y pagos sin perder la consistencia de la información.

1.2 Justificación

El sistema de comandas constituye el núcleo operativo de un salón de eventos: de él dependen la asignación de mesas, el seguimiento de los pedidos, la disponibilidad de insumos y el cobro al cliente. Un modelo de datos mal estructurado produce inconsistencias que el personal descubre hasta el cierre del evento, cuando ya no es posible corregirlas, y limita la capacidad del sistema para adaptarse a nuevos requerimientos, como los paquetes contratados, las variantes de los productos o los reportes administrativos.

Desde el punto de vista académico, el proyecto permite aplicar de forma integral los conteni-

dos de la materia sobre un caso real y conocido por el equipo: el modelado entidad-relación y su versión extendida, la transformación al modelo relacional, la definición de datos con restricciones de dominio y de integridad referencial, el control de acceso y la manipulación de datos mediante transacciones. Contar con el sistema anterior como punto de comparación facilita identificar los errores de diseño que se cometieron y comprobar que el nuevo modelo los resuelve.

Desde el punto de vista práctico, una base de datos que hace cumplir por sí misma las reglas del negocio reduce los errores en la toma de comandas, permite conocer con precisión el consumo y las existencias de insumos y entrega cuentas correctas sin conciliación manual. Además, al concentrar las reglas en el gestor y no en el programa, cualquier aplicación que se conecte a la base, ya sea de escritorio, móvil o web, queda sujeta a las mismas validaciones.

1.3 Objetivo general

Diseñar, implementar y comprobar una base de datos relacional para el sistema de comandas de un salón de eventos, que gestione de manera íntegra y consistente la información de eventos, mesas, cuentas, comandas, productos, ingredientes, personal y pagos, y que sirva de sustento a la versión profesional del sistema desarrollado previamente.

1.4 Objetivos específicos

1. Identificar los requerimientos de información del salón de eventos mediante una entrevista con el cliente y fundamentar los conceptos de bases de datos que guían el diseño.
2. Construir el modelo entidad-relación del sistema y extenderlo con los conceptos del modelo entidad-relación extendido que el caso requiere.
3. Transformar el modelo extendido en un esquema relacional mediante las reglas de transformación y ejecutarlo en PostgreSQL dentro de un contenedor de Docker.
4. Declarar los tipos de dato, dominios, restricciones y permisos que protejan las reglas del negocio desde el propio gestor.
5. Poblar la base de datos a partir de una fuente documentada, mediante un script de carga repetible y eficiente.
6. Comprobar la integridad referencial y el comportamiento de las operaciones de manipulación de datos mediante pruebas y transacciones.

1.5 Estado del arte

Para ubicar el proyecto respecto de trabajos recientes se revisaron tres estudios relacionados con los modelos de datos, la elección de un sistema gestor y la seguridad de la información. La Tabla 1 resume cada uno y su relación con el proyecto.

Referencia	Problema y metodología	Resultados principales	Relación con el proyecto
Aguilar Vera et al. (2023)	Revisión sistemática de la literatura sobre el modelado y la gestión de bases de datos NoSQL, sus tipos de datos y sus herramientas.	Identifica cuatro modelos NoSQL: clave-valor, documentos, columnas y grafos; el orientado a documentos es el más flexible y el más usado.	Permite contrastar los modelos NoSQL con el modelo relacional que exigen las cuentas, los pagos y las comandas.
Taipalus (2024)	Revisión de 117 estudios que comparan el desempeño de 44 sistemas gestores, junto con su rigor metodológico y su reproducibilidad.	Hay pocas comparaciones rigurosas y reproducibles; en general, los gestores NoSQL superan a los NewSQL en lectura y escritura.	Sustenta que la elección del gestor considere la integridad y las transacciones, y no solo el desempeño.
Machuca & Braul (2022)	Revisión de catorce artículos publicados entre 2016 y 2021 sobre las metodologías de seguridad en bases de datos.	El control de acceso es la estrategia más utilizada, seguida del cifrado y de la protección de bases de datos en la nube.	Respalda la definición de roles y permisos distintos para la aplicación, la consulta y la administración.

Tabla 1
Trabajos relacionados con el proyecto

Los tres trabajos son revisiones sistemáticas y coinciden en que ninguna tecnología de bases de datos es superior en términos absolutos: la elección depende de las necesidades del sistema. Taipalus (2024) reporta que los gestores NoSQL superan a otras familias en velocidad de lectura y escritura; sin embargo, en un sistema de comandas una cuenta que no cuadra o un pago registrado dos veces afectan directamente la operación del salón, por lo que la integridad y la consistencia pesan más que la velocidad. Por esa razón se eligió PostgreSQL, un gestor relacional con soporte completo de transacciones y restricciones de integridad. De igual forma, lo reportado por Machuca y Braul (2022) sobre el control de acceso se tradujo en roles con el mínimo privilegio, de modo que cada usuario de la base pueda realizar únicamente las operaciones que le corresponden.

1.6 Alcance

El documento abarca la base de datos del sistema de comandas en todas sus etapas: el análisis del caso de estudio, el diseño conceptual con los modelos entidad-relación y entidad-relación extendido, el diseño lógico con el modelo relacional, la implementación física en PostgreSQL, la población con datos y las pruebas de integridad y de manipulación de datos. Cada etapa corresponde a una o varias de las prácticas de la materia, y su documentación detallada, junto con los scripts y las evidencias, se encuentra en el repositorio del equipo.

Quedan fuera de este documento la aplicación que utiliza la base de datos, cuyo cliente y servidor se desarrollan por separado, y el proyecto asignado por la materia, que se documenta de forma independiente.

1.7 Organización del documento

Después de esta introducción, el marco teórico reúne los conceptos que sustentan el diseño y la sección de metodología describe la forma de trabajo y el entorno en que se desarrolló el proyecto. Enseguida se presentan el caso de estudio con sus requerimientos, el diseño conceptual y el diseño lógico de la base de datos. Las secciones siguientes describen su implementación en PostgreSQL, la población con datos y las pruebas de integridad referencial y de manipulación de datos. Por último, se presentan los resultados y las conclusiones del proyecto.

2 Marco teórico

En esta sección se reúnen los conceptos que sustentan el diseño y la implementación de la base de datos: la noción de base de datos y de sistema gestor, las etapas de su ciclo de vida, los modelos con los que se representó el caso de estudio y los mecanismos del lenguaje SQL con los que el gestor hace cumplir las reglas del negocio.

2.1 Dato, información y base de datos

Un dato es un hecho que puede ser registrado y que tiene un significado implícito (Elmasri & Navathe, 2016, p. 4); por sí solo no tiene un significado más allá de su propia existencia (Bellinger et al., 2004, p. 1). La información surge cuando los datos se relacionan entre sí y adquieren significado a través de esa conexión (Bellinger et al., 2004, p. 1). Por ejemplo, el número de una mesa no dice nada aislado, pero ligado a un evento, a un mesero y a una cuenta permite saber quién atendió a qué grupo de invitados y cuánto consumió.

Una base de datos es, en consecuencia, una colección de datos relacionados de manera lógica y coherente, que representa algún aspecto del mundo real y que se diseña y se puebla con un propósito específico (Elmasri & Navathe, 2016, p. 5). Para Date (2001), es un sistema computarizado cuyo objetivo es llevar el registro de la información y permitir que los usuarios la consulten y la actualicen (p. 2).

2.2 Sistema gestor de bases de datos

Un sistema gestor de bases de datos (SGBD) es el conjunto de programas que facilita los procesos de definir, construir, manipular y compartir bases de datos entre diversos usuarios y aplicaciones (Elmasri & Navathe, 2016, p. 6). Su función es mediar entre los archivos físicos y quienes usan la información, de manera que las aplicaciones no dependan de la forma en que los datos se almacenan. Frente a los sistemas de archivos tradicionales, en los que cada programa define y mantiene sus propios archivos, un SGBD concentra la definición de los datos en un catálogo y ofrece las características que se resumen en la Tabla 2.

Característica	Descripción
Integración	Los datos se concentran en un solo repositorio en lugar de archivos independientes e inconexos.
Persistencia	Los datos sobreviven a los programas que los crearon y solo cambian por una petición al gestor.
Redundancia controlada	La duplicación se evita mediante la normalización y, cuando existe a propósito, el gestor mantiene las copias sincronizadas.
Integridad	Los datos cumplen las restricciones declaradas en el catálogo, que el gestor valida en cada actualización.
Independencia de datos	El esquema físico o lógico puede cambiar sin modificar los programas que acceden a los datos.
Seguridad	Solo los usuarios autorizados pueden leer o modificar cada objeto, según los privilegios que se les otorgan.
Concurrencia	Varios usuarios operan al mismo tiempo sin producir lecturas sucias ni actualizaciones perdidas.
Recuperación	Tras una falla, la base regresa a un estado consistente que conserva las transacciones confirmadas.

Tabla 2
Características de un sistema gestor de bases de datos

Estas características se ejercen mediante cuatro sublenguajes que, en los gestores basados en SQL, conviven en una misma interfaz (Elmasri & Navathe, 2016, pp. 38–41; Silberschatz et al., 2002, pp. 42–44): el lenguaje de definición de datos (DDL), con el que se crean tablas, dominios y restricciones; el lenguaje de manipulación de datos (DML), con el que se insertan, modifican, borran y consultan los datos; el lenguaje de control de datos (DCL), que administra los privilegios de los usuarios; y el lenguaje de control de transacciones (TCL), que delimita las unidades de trabajo que se confirman o se descartan por completo.

Para separar las aplicaciones de los detalles de almacenamiento, los gestores siguen la arquitectura de tres niveles propuesta por el comité ANSI/SPARC (Elmasri & Navathe, 2016, pp. 36–37): el nivel interno describe cómo se almacenan los datos, el nivel conceptual describe la estructura lógica de toda la base y el nivel externo ofrece vistas con la porción de datos que interesa a cada grupo de usuarios. Gracias a esta separación, un cambio en un nivel no obliga a modificar el nivel superior ni los programas de aplicación (Elmasri & Navathe, 2016, pp. 37–38).

2.3 Ciclo de vida de una base de datos

Una base de datos real tiene un ciclo de vida de muchos años, durante el cual debe evolucionar conforme cambian los requerimientos (Elmasri & Navathe, 2016, p. 8). Su desarrollo se divide en etapas que van de la idea del negocio a la operación, y cada una produce un entregable que sirve de entrada a la siguiente. Este proyecto siguió esas etapas, que se describen a continuación junto con la práctica en que se trabajó cada una.

1. **Recolección y análisis de requerimientos.** Se documenta qué información necesita la organización y qué operaciones realiza sobre ella (Elmasri & Navathe, 2016, p. 8). En el

proyecto corresponde a la entrevista con el cliente de la Práctica 1.

2. **Diseño conceptual.** Los requerimientos se expresan en un modelo semántico independiente del gestor, como el modelo entidad-relación (Chen, 1976, pp. 9–10). Se realizó en las prácticas 1 y 2.

3. **Diseño lógico.** El esquema conceptual se traduce al modelo del gestor elegido, por lo general el relacional, mediante reglas de transformación (Elmasri & Navathe, 2016, p. 289). Corresponde a la Práctica 3.

4. **Diseño físico e implementación.** El esquema lógico se escribe en sentencias DDL, se ejecuta en el gestor y la base se puebla con los datos iniciales (Date, 2001, p. 41; Elmasri & Navathe, 2016, p. 31). Se realizó en las prácticas 3 y 4.

5. **Pruebas y operación.** Se comprueba que la base proteja sus reglas y se pone en uso mediante las aplicaciones (Elmasri & Navathe, 2016, pp. 43–45). Las pruebas corresponden a las prácticas 4 y 5.

2.4 Modelo entidad-relación y su extensión

El modelo entidad-relación, propuesto por Chen (1976), representa el mundo real mediante entidades, que son los objetos de interés; atributos, que son sus propiedades; y relaciones, que son las asociaciones entre entidades. Un atributo puede ser simple o compuesto, monovaluado o multivaluado, y almacenado o derivado, cuando su valor se calcula a partir de otros datos (Elmasri & Navathe, 2016). Cada relación tiene un grado, que es el número de entidades que participan en ella, y restricciones estructurales que se expresan con la notación (min, max): el mínimo indica si la participación de una entidad es obligatoria y el máximo, con cuántas instancias de la otra entidad puede asociarse (Elmasri & Navathe, 2016).

Una entidad débil es aquella que no tiene una clave propia suficiente para identificarse y depende de una entidad propietaria a través de una relación identificadora; su clave se forma con la clave de la propietaria y una clave parcial (Elmasri & Navathe, 2016). Por ejemplo, la mesa número tres solo se distingue si se sabe a qué evento pertenece.

El modelo entidad-relación extendido (EER) añade los conceptos de especialización y generalización, con los que una entidad general, o superclase, se divide en subclases que heredan sus atributos y agregan los propios (Elmasri & Navathe, 2016). Una especialización está sujeta a dos restricciones: de disyunción, que indica si una instancia puede pertenecer a una sola subclase (disjunta) o a varias (solapada), y de completitud, que indica si toda instancia de la superclase debe pertenecer a alguna subclase (total) o no (parcial). El modelo extendido también contempla relaciones de grado superior, como las ternarias, que se justifican cuando ningún par de las entidades participantes determina a la tercera.

2.5 Modelo relacional

El modelo relacional representa la base de datos como una colección de relaciones, o tablas, en las que cada fila es una tupla y cada columna es un atributo cuyo valor pertenece a un dominio, es decir, a un conjunto de valores atómicos permitidos (Elmasri & Navathe, 2016). Una clave candidata es un conjunto mínimo de atributos que identifica de forma única a cada tupla; la clave primaria es la candidata que se elige para ese fin, y una clave foránea es un conjunto de atributos que hace referencia a la clave primaria de otra relación.

Sobre estas estructuras se definen las restricciones de integridad del modelo (Elmasri & Navathe, 2016): la restricción de dominio exige que cada valor pertenezca al dominio de su atributo; la de clave, que no existan dos tuplas con la misma clave; la de integridad de entidad, que la clave primaria nunca sea nula; y la de integridad referencial, que toda clave foránea haga referencia a una tupla que exista.

El paso del modelo extendido al relacional se realiza con un conjunto de reglas de transformación (Elmasri & Navathe, 2016, p. 289). Las reglas que se aplicaron en el proyecto convierten cada entidad fuerte en una tabla; descomponen los atributos compuestos en sus componentes; llevan cada atributo multivaluado a una tabla propia; transforman las relaciones uno a muchos en una clave foránea del lado de muchos, las de muchos a muchos en una tabla intermedia y las de uno a uno en una clave foránea única; convierten las relaciones de grado superior en una tabla cuya clave combina las claves de las entidades participantes; forman la clave de cada entidad débil con la de su propietaria; y representan las jerarquías con una de varias estrategias, como una tabla por superclase y una por subclase.

2.6 Implementación en un gestor relacional

Los conceptos anteriores se hacen cumplir en el gestor mediante las sentencias de SQL. En esta sección se describen los mecanismos de PostgreSQL que se utilizaron en el proyecto.

2.6.1 Restricciones de dominio

Al definir una tabla, cada columna recibe un tipo de dato y, opcionalmente, restricciones que acotan sus valores (PostgreSQL Global Development Group, s.f.-a): `NOT NULL` impide que la columna quede vacía; `DEFAULT` asigna un valor inicial cuando no se especifica uno; `UNIQUE` impide que dos filas repitan el mismo valor o combinación de valores; y `CHECK` exige que cada fila cumpla una condición, ya sea un rango, una lista de valores permitidos, un formato o una comparación entre columnas. Cuando un mismo tipo y una misma precisión se repiten en varias tablas, la sentencia `CREATE DOMAIN` permite definirlos una sola vez con un nombre propio y reutilizarlos (PostgreSQL Global Development Group, s.f.-c).

2.6.2 Integridad referencial

Una clave foránea se declara con `FOREIGN KEY` y hace que el gestor rechace cualquier fila que apunte a un renglón inexistente (PostgreSQL Global Development Group, s.f.-a). La declaración incluye además una acción referencial, que determina qué ocurre con las filas hijas cuando se borra o se actualiza la fila padre: `RESTRICT` impide la operación mientras existan hijas; `CASCADE` la propaga a las hijas; `SET NULL` deja la referencia en nulo; y `SET DEFAULT` la reemplaza por un valor por omisión. La elección depende de la regla del negocio: `CASCADE` corresponde cuando la hija no tiene sentido sin su padre, como una entidad débil, y `RESTRICT` cuando el padre no debe desaparecer mientras algo dependa de él.

2.6.3 Disparadores y vistas

Algunas reglas del negocio relacionan varias filas o varias tablas y no pueden expresarse con una restricción de columna. Para ellas se emplean disparadores, que son funciones que el gestor ejecuta automáticamente antes o después de una inserción, actualización o borrado, y que pueden rechazar la operación o modificar los datos (PostgreSQL Global Development Group, s.f.-i). Por su parte, una vista es una consulta almacenada con un nombre, que se consulta como si fuera una tabla (PostgreSQL Global Development Group, s.f.-d); se utiliza para calcular atributos derivados sin guardarlos, de modo que su valor nunca queda desactualizado, y para ofrecer a cada grupo de usuarios solo la porción de datos que necesita.

2.6.4 Transacciones

Una transacción es una unidad de trabajo que agrupa varias operaciones y que debe ejecutarse completa o no ejecutarse (PostgreSQL Global Development Group, s.f.-h). Las transacciones cumplen las propiedades conocidas como ACID: atomicidad, porque sus operaciones se aplican todas o ninguna; consistencia, porque llevan la base de un estado válido a otro; aislamiento, porque su ejecución no se ve afectada por otras transacciones concurrentes; y durabilidad, porque sus cambios confirmados persisten aun ante fallas (Elmasri & Navathe, 2016). En SQL, una transacción inicia con `BEGIN`, se confirma con `COMMIT` y se descarta con `ROLLBACK`. Dentro de ella, `SAVEPOINT` marca un punto de recuperación al que es posible regresar con `ROLLBACK TO` sin descartar el trabajo anterior (PostgreSQL Global Development Group, s.f.-g).

2.6.5 Control de acceso

El acceso a la base se administra mediante roles, que pueden representar a un usuario o a un grupo de usuarios (PostgreSQL Global Development Group, s.f.-e). Con `GRANT` se otorgan privilegios sobre cada objeto, como leer, insertar, modificar o borrar en una tabla, y con `REVOKE` se retiran (PostgreSQL Global Development Group, s.f.-f). El principio de mínimo privilegio con-

siste en conceder a cada rol únicamente los permisos que su función requiere, de modo que un error en una aplicación o un acceso indebido no pueda afectar más datos de los necesarios.

2.6.6 Carga masiva de datos

Para poblar una base con volúmenes grandes de datos, insertar cada fila con una sentencia `INSERT` resulta lento, porque cada sentencia se procesa por separado. La sentencia `COPY` de PostgreSQL, y su variante `\copy` del cliente `psql`, transfiere el contenido completo de un archivo a una tabla en una sola operación (PostgreSQL Global Development Group, s.f.-b), por lo que es el mecanismo recomendado para la carga inicial de una base de datos.

3 Metodología y entorno de trabajo

En esta sección se describe la forma en que se organizó el desarrollo del proyecto, las herramientas con las que el equipo coordinó su trabajo y el entorno en que se ejecutó el gestor de bases de datos.

3.1 Desarrollo por etapas

El proyecto se desarrolló siguiendo las etapas del ciclo de vida de una base de datos descritas en el marco teórico. Cada práctica de la materia correspondió a una o varias de esas etapas, y su resultado fue la entrada de la práctica siguiente, como se resume en la Tabla 3.

Práctica	Etapa	Productos
1	Requerimientos y diseño conceptual	Marco teórico, entrevista con el cliente, requerimientos, diagrama entidad-relación y entorno de trabajo con Git y Docker.
2	Diseño conceptual	Modelo entidad-relación extendido en notación de Peter Chen y análisis de los artículos del proyecto asignado.
3	Diseño lógico e implementación	Inventario del modelo extendido, reglas de transformación, esquema relacional ejecutado en PostgreSQL, diccionario de datos y pruebas de las restricciones.
4	Implementación y población	Tipos de dato y dominios, restricciones de columna, fuente de datos, script de carga, pruebas de dominio, usuarios y permisos.
5	Pruebas de integridad	Claves foráneas y acciones referenciales, pruebas de integridad, edición de valores, transacciones y experimentos individuales.

Tabla 3
Etapas del ciclo de vida trabajadas en cada práctica

El desarrollo no fue estrictamente lineal. Cada etapa permitió detectar omisiones de la anterior y corregirlas antes de continuar: al construir el modelo extendido se reconsideraron los requerimientos de la entrevista; al transformar el modelo al relacional se levantó primero un inventario de entidades, relaciones y atributos especiales; y al implementar las restricciones de dominio se encontraron reglas que el esquema todavía no protegía. A partir de la Práctica 4, todo cambio al esquema ya ejecutado se registró como una migración numerada con sentencias `ALTER TABLE`, en lugar de editar el script original, de modo que la historia del esquema quedara documentada y pudiera reproducirse en cualquier base.

3.2 Control de versiones

El código, la documentación y las evidencias del proyecto se administran con Git en un repositorio compartido en GitHub. Git registra cada cambio como una confirmación con su autor y su

fecha, lo que permite reconstruir la historia del proyecto y recuperar cualquier versión anterior (Chacon & Straub, 2014); GitHub agrega un repositorio remoto común y la revisión de cambios mediante *pull requests* (GitHub, s.f.-b).

El equipo adoptó un flujo de trabajo basado en ramas (Atlassian, s.f.), representado en la Figura 1. La rama `main` contiene únicamente el trabajo revisado y entregado. Al iniciar una práctica se crea a partir de ella una rama de práctica, y cada sección de la práctica se desarrolla en una rama propia que sale de la rama de práctica. Al terminar una sección, sus cambios se integran a la rama de práctica mediante un *pull request*, y al cerrar la práctica, la rama de práctica se integra a `main` con un *pull request* revisado por otro integrante del equipo.

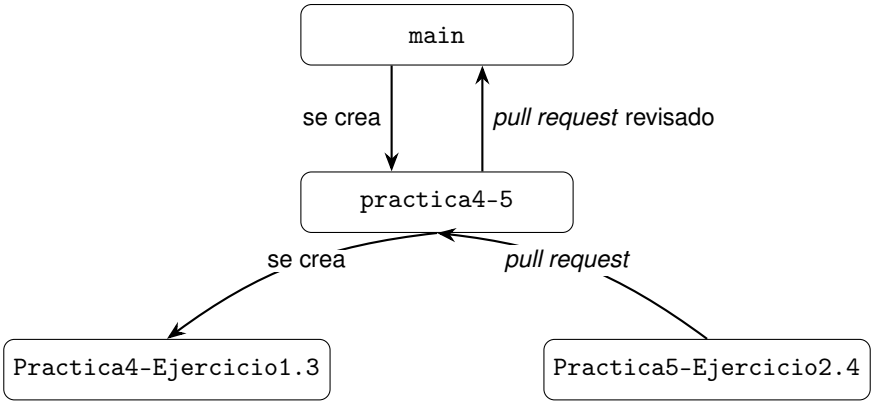


Figura 1
Flujo de ramas del repositorio, con ejemplos de las prácticas 4 y 5

La revisión entre pares antes de cada integración permite detectar errores, proteger la rama principal y compartir el conocimiento entre los integrantes (GitHub, s.f.-a). A lo largo del proyecto se integraron cerca de cuarenta *pull requests*, y cada integrante registró sus propias confirmaciones, de modo que el historial del repositorio funciona también como evidencia de la participación de cada uno.

3.3 Contenedores y gestor de bases de datos

El gestor de bases de datos se ejecuta en un contenedor de Docker. Un contenedor empaqueta una aplicación con todo lo que necesita para funcionar y la aísla del sistema anfitrión, de manera que se comporta igual en cualquier equipo (Docker, s.f.-c). Esto resuelve un problema habitual en el trabajo en equipo: que la base funcione en la computadora de un integrante y no en la de otro por diferencias de versión o de configuración.

La definición del contenedor se encuentra en un archivo de Docker Compose que forma parte del repositorio y que se ha reutilizado sin cambios desde la Práctica 1. Sus parámetros principales se presentan en la Tabla 4.

Parámetro	Valor
Gestor	PostgreSQL 16.4, sobre la imagen oficial `postgres:16.4-alpine`
Contenedor	`practica1-postgres`
Puerto publicado	`5432` en el anfitrión y en el contenedor
Volumen	`practica1-pgdata`, montado en el directorio de datos del gestor
Codificación	UTF-8
Zona horaria	`America/Mexico_City`
Credenciales	Variables de entorno, con valores por omisión para el entorno de desarrollo

Tabla 4
Configuración del contenedor del gestor

Los datos de un contenedor se pierden cuando este se elimina, salvo que se guarden en un volumen, que es un espacio de almacenamiento administrado por Docker e independiente del ciclo de vida del contenedor (Docker, s.f.-b). Por ello, el directorio de datos de PostgreSQL se monta en un volumen con nombre. En la Práctica 1 se comprobó que la base y sus datos se conservan después de eliminar y volver a crear el contenedor. El puerto publicado permite, además, que las herramientas del equipo anfitrión, como el cliente `psql` o un administrador gráfico, se conecten al gestor como si estuviera instalado localmente (Docker, s.f.-a).

3.4 Automatización y organización del repositorio

Para que cualquier integrante pudiera reproducir el estado de la base y verificar que las reglas del esquema se cumplen, las tareas repetitivas se automatizaron con un archivo `Makefile`. Un solo comando levanta el contenedor, recrea una base de pruebas, ejecuta el esquema, las migraciones y los datos de ejemplo, y corre todas las pruebas: las instrucciones que deben ser rechazadas, los casos válidos y las pruebas de integridad referencial. Si alguna prueba no da el resultado esperado, el proceso se detiene e informa cuál falló. La carga de la base completa, las pruebas de permisos y los ejercicios de manipulación de datos cuentan con sus propios scripts, que leen las credenciales de variables de entorno y nunca las guardan en el repositorio.

El repositorio se organiza por tipo de contenido, como se muestra en la Tabla 5. Además, desde la carpeta `web` se publica un sitio del proyecto mediante GitHub Pages.

Carpeta	Contenido
`docs`	Documentación de cada práctica en Markdown, enunciados y este documento en LaTeX
`sql`	Esquema, migraciones, pruebas, script de carga, datos y sentencias de la Práctica 5
`modelo`	Diagramas entidad-relación, entidad-relación extendido y relacional
`evidencias`	Capturas y salidas de la terminal de cada práctica
`entorno`	Definición del contenedor del gestor
`web`	Sitio del proyecto

Tabla 5
Organización del repositorio

Referencias

Aguilar Vera, R., Naal Jácome, A., Díaz Mendoza, J., & Gómez Gómez, O. (2023). NoSQL database modeling and management: A systematic literature review. *Revista Facultad de Ingeniería*, *32*(65), Artículo e16519. https://doi.org/10.19053/01211129.v32.n65.2023.16519

Atlassian. (s.f.). *Git feature branch workflow*. Atlassian Git Tutorials. Consultado el 16 de septiembre de 2026, desde https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow

Bellinger, G., Castro, D., & Mills, A. (2004). *Data, information, knowledge, and wisdom*. Universidade Federal de Minas Gerais (UFMG). Consultado el 18 de septiembre de 2026, desde https://homepages.dcc.ufmg.br/~amendes/SistemasInformacaoTP/TextosBasicos/Data-Information-Knowledge.pdf

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Consultado el 16 de septiembre de 2026, desde https://git-scm.com/book/en/v2

Chen, P. P.-S. (1976). The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, *1*(1), 9-36. https://doi.org/10.1145/320434.320440

Date, C. J. (2001). *Introducción a los sistemas de bases de datos* (S. L. M. Ruíz Faudón, Trad.; 7.ª ed.). Pearson Educación.

Docker. (s.f.-a). *Publishing and exposing ports*. Docker Docs. Consultado el 16 de septiembre de 2026, desde https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/

Docker. (s.f.-b). *Volumes*. Docker Docs. Consultado el 16 de septiembre de 2026, desde https://docs.docker.com/engine/storage/volumes/

Docker. (s.f.-c). *What is a container?* Docker Docs. Consultado el 16 de septiembre de 2026, desde https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

GitHub. (s.f.-a). *About pull request reviews*. GitHub Docs. Consultado el 16 de septiembre de 2026, desde https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

GitHub. (s.f.-b). *About repositories*. GitHub Docs. Consultado el 16 de septiembre de 2026, desde https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

Machuca, L., & Braul, R. (2022). Metodologías más usadas en la seguridad de bases de datos: Una revisión de la literatura científica, 2016–2021. *Campus*, *27*(33), 121-130. https://doi.org/10.24265/campus.2022.v27n33.08

PostgreSQL Global Development Group. (s.f.-a). *Constraints*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/ddl-constraints.html

PostgreSQL Global Development Group. (s.f.-b). *COPY*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-copy.html

PostgreSQL Global Development Group. (s.f.-c). *CREATE DOMAIN*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-createdomain.html

PostgreSQL Global Development Group. (s.f.-d). *CREATE VIEW*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-createview.html

PostgreSQL Global Development Group. (s.f.-e). *Database roles*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/database-roles.html

PostgreSQL Global Development Group. (s.f.-f). *Privileges*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/ddl-priv.html

PostgreSQL Global Development Group. (s.f.-g). *SAVEPOINT*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-savepoint.html

PostgreSQL Global Development Group. (s.f.-h). *Transactions*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/tutorial-transactions.html

PostgreSQL Global Development Group. (s.f.-i). *Triggers*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/triggers.html

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2002). *Database system concepts: Instructor's manual* (4.ª ed.). McGraw-Hill Higher Education.

Taipalus, T. (2024). Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, *208*, Artículo 111872. https://doi.org/10.1016/j.jss.2023.111872